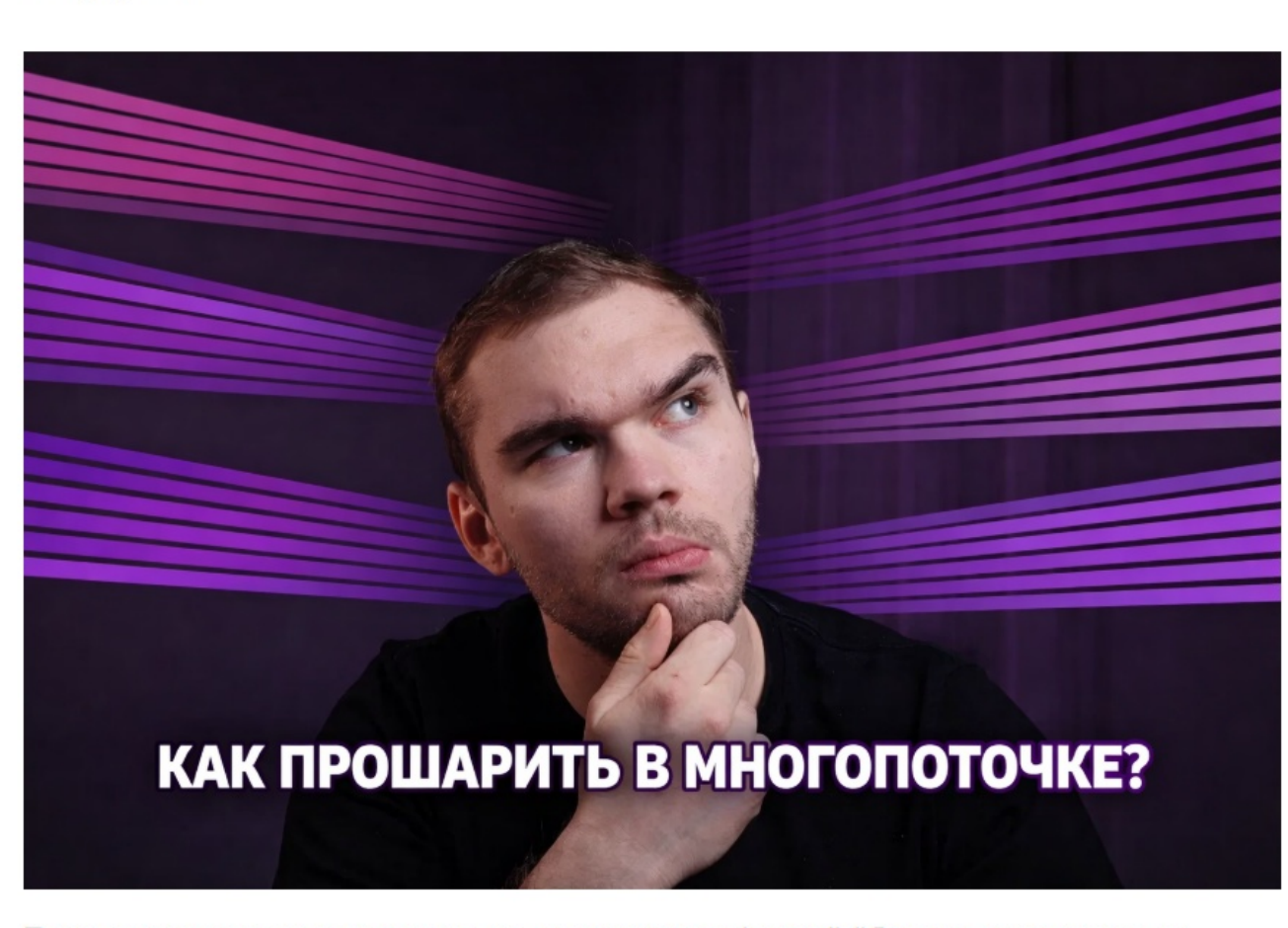
Павел Сорокин
@pavel.sorokin
Подписаться

5 марта

Как разобраться в многопоточке от и до?


КАК ПРОШАРИТЬ В МНОГОПОТОЧКЕ?

Тема многопоточности идеально описывается фразой *"Я знаю, что ничего не знаю"*

Чем больше ты её изучаешь, тем сильнее осознаёшь как сильно тебя обманывали все предыдущие годы

Обманывал универ. Обманывали статьи. Обманывали видосы

Поэтому в этой статье я хочу провести небольшое расследование. Предмет расследования: **"Почему миддл с тремя годами опыта молчит на собесе как рыба, когда его спрашивают базу по потокам?"**, а конкретнее:

ЧТО, В КАКОМ ПОРЯДКЕ И ДО КАКОЙ ГЛУБИНЫ ИЗУЧАТЬ В ТЕМЕ МНОГОПОТОЧНОСТИ?

Тема щепетильная. У некоторых разработчиков от слова "многопоточка" начинается лёгкая паника и желание срочно вспомнить что у них было что-то важное по другой задаче

Поэтому сразу предупрежу: всё что тут написано основано на моём личном опыте и сотнях часов сбора и систематизации знаний

Ещё тут будет несколько факапов с моих работ разработчиком в продакшене. Приятного чтения:)

Глава 1. Многопоточка нужна всем, многопоточку не знает никто

@julianmallog5262
"knock knock"
"race condition"
"who's there?"

Я до каких-то пор думал что все джависты хотя бы примерно понимают многопоточку. Ну я же сам понимаю - значит и все понимают *(да, знаю, логика железная)*

> Длилось это ровно до того момента, как я начал проводить мок-собесы для своего YouTube-канала

Сажусь с чуваком. Мидл-разработчик, три года реального опыта. Человек, который уже работал, получал деньги, закрывал таски

Спрашиваю самую базу из базы: *"Объясни разницу между процессом и потоком"*

Он такой: *"Эээ... ну там ядра короче есть... и там ну вот ОС сама решает какой поток типа отдать посчитать..."*

Как вам ответ? Передали бы кандидата на месте HR'a к техлиду?

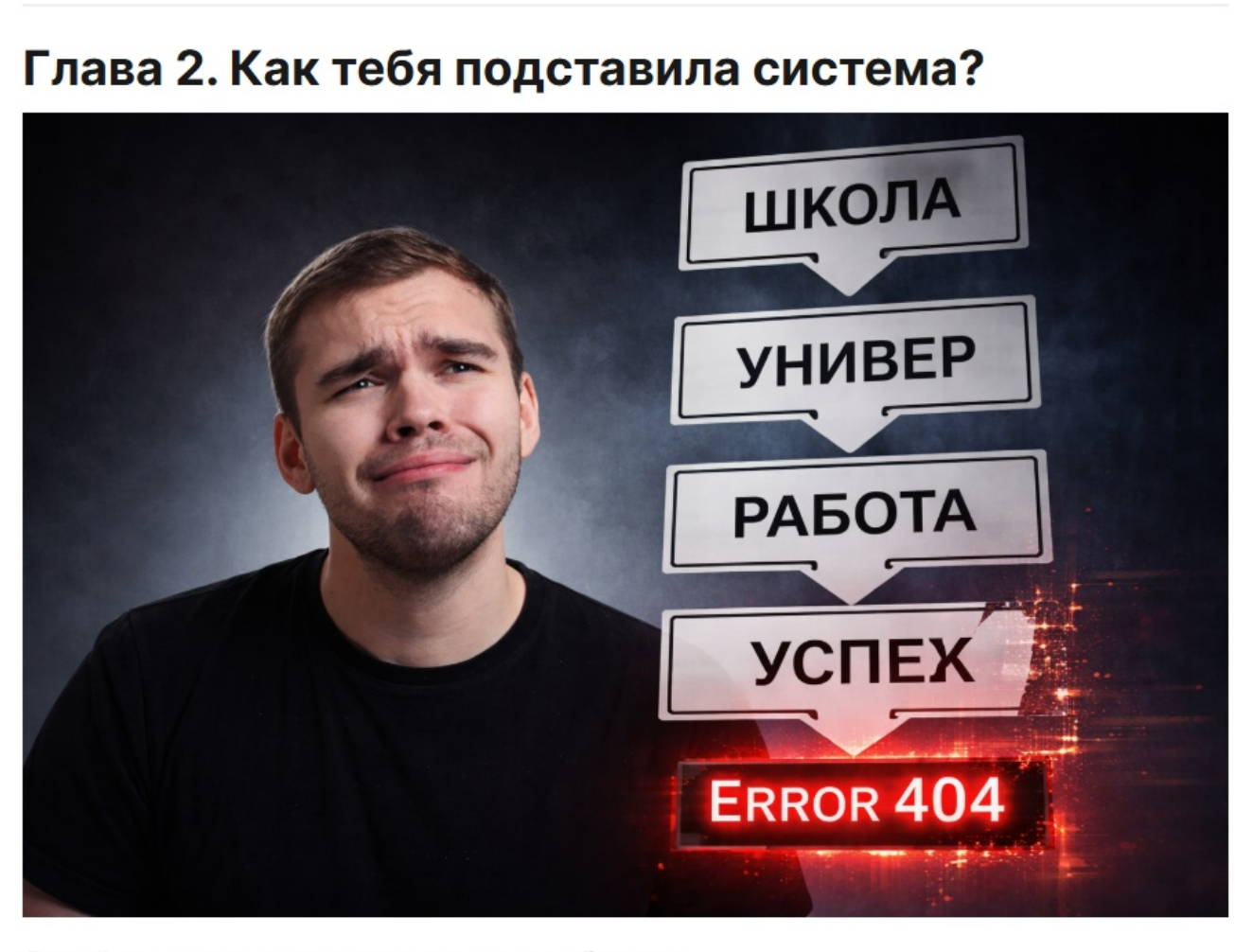
Провожу второй собес с джуном. На любом вопросе по многопоточке просто молчал. Там реальный deadlock в голове случился и он просто молчал, попросив на монтаже вырезать этот кусок *(спойлер: это не его вина)*

И знаете что **самое интересное?**

Так у ВСЕХ. Или почти у всех. Я не преувеличиваю

Только это не вина разработчиков. Это структурная подстава рынка, которую почему-то не принято обсуждать вслух

Глава 2. Как тебя подставила система?

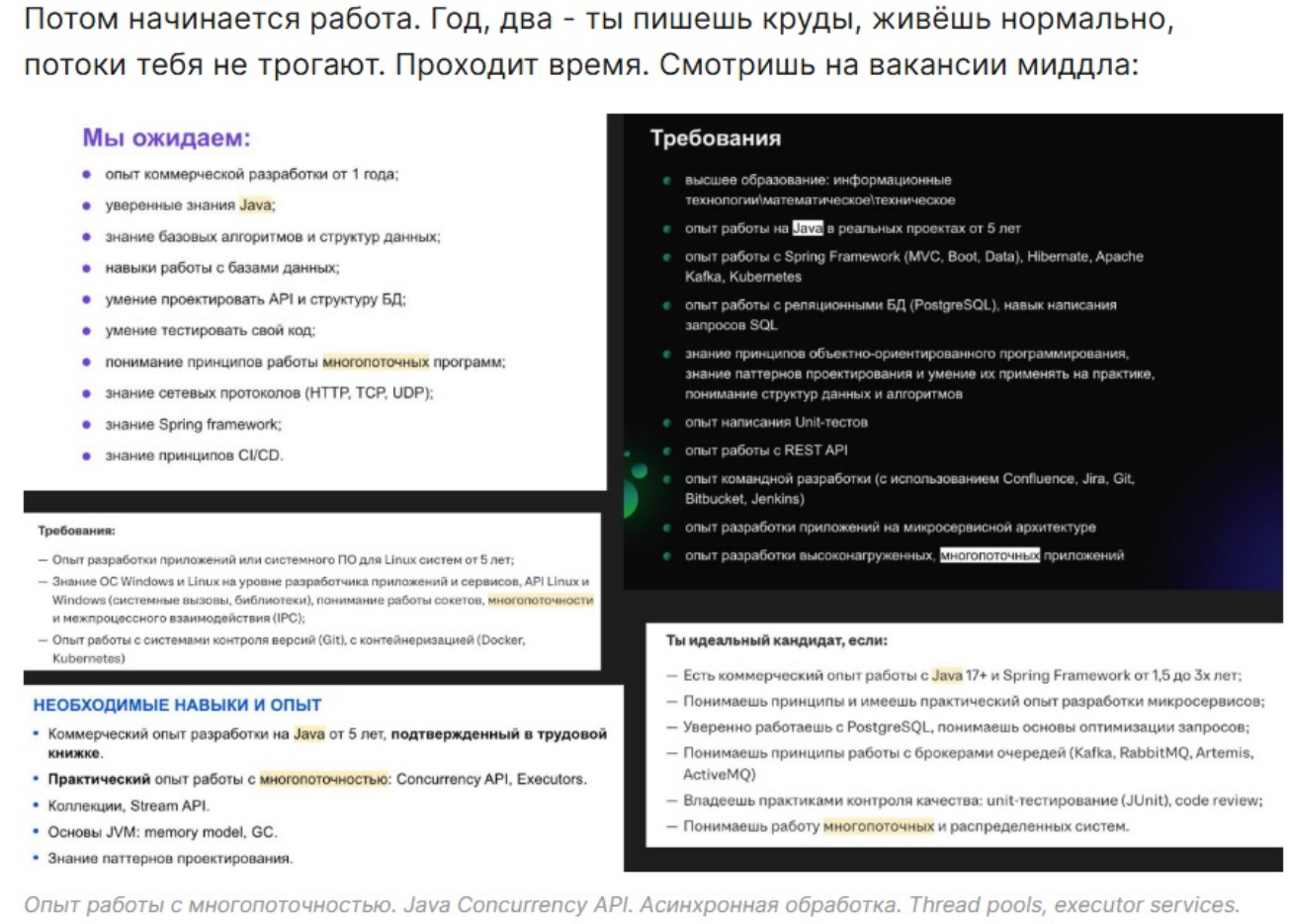

ШКОЛА
УНИВЕР
РАБОТА
УСПЕХ
ERROR 404

Давайте вместе посмотрим как это работает:

В универах многопоточку либо не преподают вообще, либо дают такую академическую духоту, что после пары мозг превращается в кашу

Профессор рисует квадратики с L1 и L2 кэшами, бубнит про регистры. Половина аудитории в телефонах. Ты сидишь, киваешь, **ничего не понимаешь**, сдаёшь экзамен по билетам и выходишь с ощущением что "прошли тему"

Потом начинается работа. Год, два - ты пишешь круды, живёшь нормально, потоки тебя не трогают. Проходит время. Смотришь на вакансии миддла:



Мы ожидаем:

- опыт коммерческой разработки от 1 года;
- уверенные знания Java;
- знание базовых алгоритмов и структур данных;
- навыки работы с базами данных;
- умение проектировать API и структуру БД;
- умение тестировать свой код;
- понимание принципов работы многопоточных программ;
- знание сетевых протоколов (HTTP, TCP, UDP);
- знание Spring framework;
- знание принципов CI/CD.

Требования

- высшее образование: информационные технологии\математическое\техническое
- опыт работы на Java в реальных проектах от 5 лет
- опыт работы с Spring Framework (MVC, Boot, Data), Hibernate, Apache Kafka, Kubernetes
- опыт работы с реляционными БД (PostgreSQL), навык написания запросов SQL
- знание принципов объектно-ориентированного программирования, знание паттернов проектирования и умение их применять на практике, понимание структур данных и алгоритмов
- опыт написания Unit-тестов
- опыт работы с REST API
- опыт командной разработки (с использованием Confluence, Jira, Git, Bitbucket, Jenkins)
- опыт разработки приложений на микросервисной архитектуре
- опыт разработки высоконагруженных, многопоточных приложений

Требования:

- Опыт разработки приложений или системного ПО для Linux систем от 5 лет;
- Знание ОС Windows и Linux на уровне разработчика приложений и сервисов, API Linux и Windows (системные вызовы, библиотеки), понимание работы сокетов, многопоточности и межпроцессного взаимодействия (IPC);
- Опыт работы с системами контроля версий (Git), с контейнеризацией (Docker, Kubernetes)

НЕОБХОДИМЫЕ НАВЫКИ И ОПЫТ

- Коммерческий опыт разработки на Java от 5 лет, **подтвержденный в трудовой книжке.**
- **Практический** опыт работы с многопоточностью: Concurrency API, Executors.
- Коллекции, Stream API.
- Основы JVM: memory model, GC.
- Знание паттернов проектирования.

Ты идеальный кандидат, если:

- Есть коммерческий опыт работы с Java 17+ и Spring Framework от 1,5 до 3х лет;
- Понимаешь принципы и имеешь практический опыт разработки микросервисов;
- Уверенно работаешь с PostgreSQL, понимаешь основы оптимизации запросов;
- Понимаешь принципы работы с брокерами очередей (Kafka, RabbitMQ, Artemis, ActiveMQ)
- Владеешь практиками контроля качества: unit-тестирование (JUnit), code review;
- Понимаешь работу многопоточных и распределенных систем.

Опыт работы с многопоточностью. Java Concurrency API. Асинхронная обработка. Thread pools, executor services.

И ты думаешь: *"Ну вроде слышал что-то такое..."*

Вот тут начинается развилка:

- **Первый вариант** - ты просто продолжаешь игнорировать тему, потому что непонятно как её вообще учить. Берёшь рандомные статьи, открываешь, читаешь про `synchronized`, потом видишь отсылку на JMM, открываешь JMM, там happens-before, гуглишь happens-before - и через два часа у тебя 47 вкладок, полный туман в голове и желание закрыть ноут и пойти поспать
- **Второй вариант** - ты как-то доходишь до собеса и плывёшь на нетривиальном вопросе

Ладно, бывает ещё хуже - ты проходишь на работу, а потом заваливаешь испытательный срок, потому что тебя посадили разбираться с багами, где внутри надо понимать потоки:)))

Это буквально две реальные истории моих студентов, которые пришли ко мне именно после такого, потому что уж очень им было обидно чувствовать себя самозванцами, так ещё и недозарабатывать из-за одной темы...

Глава 3. WHERE'S THE MONEY, разработчик?

РЫНОК НАЙМА
ТЫ

Давай по-честному посчитаем во сколько тебе обходится незнание этой темы

- Джун без понимания concurrency - **100-150к в месяц**
- Миддл, который шарит в многопоточке, понимает асинхронность, умеет проектировать системы - **200-300к**

> Разница +- 100к в месяц. Это 1.200.000 рублей в год. **За три года - 3.600.000 рублей, которые ты просто не заработал**

Потому что не выучил одну тему и не походил по собесам, либо провалил их

Кто-то сейчас скажет: *"Ну зарплата зависит не только от многопоточки"*

Справедливо. Но вот что точно зависит: твоя уверенность на собесах, твоя репутация на работе, и то насколько быстро тебя зовут разбирать инциденты - или наоборот не зовут совсем

Ещё грустнее то, что незнание этой темы запускает **синдром самозванца**. Ты чувствуешь что что-то не понимаешь, боишься спросить и избегаешь задач, которые могут вскрыть пробел. И так годами

Я сам через это прошёл. Причём дважды с публичным позором

Глава 4. Как я облажался. Дважды:)

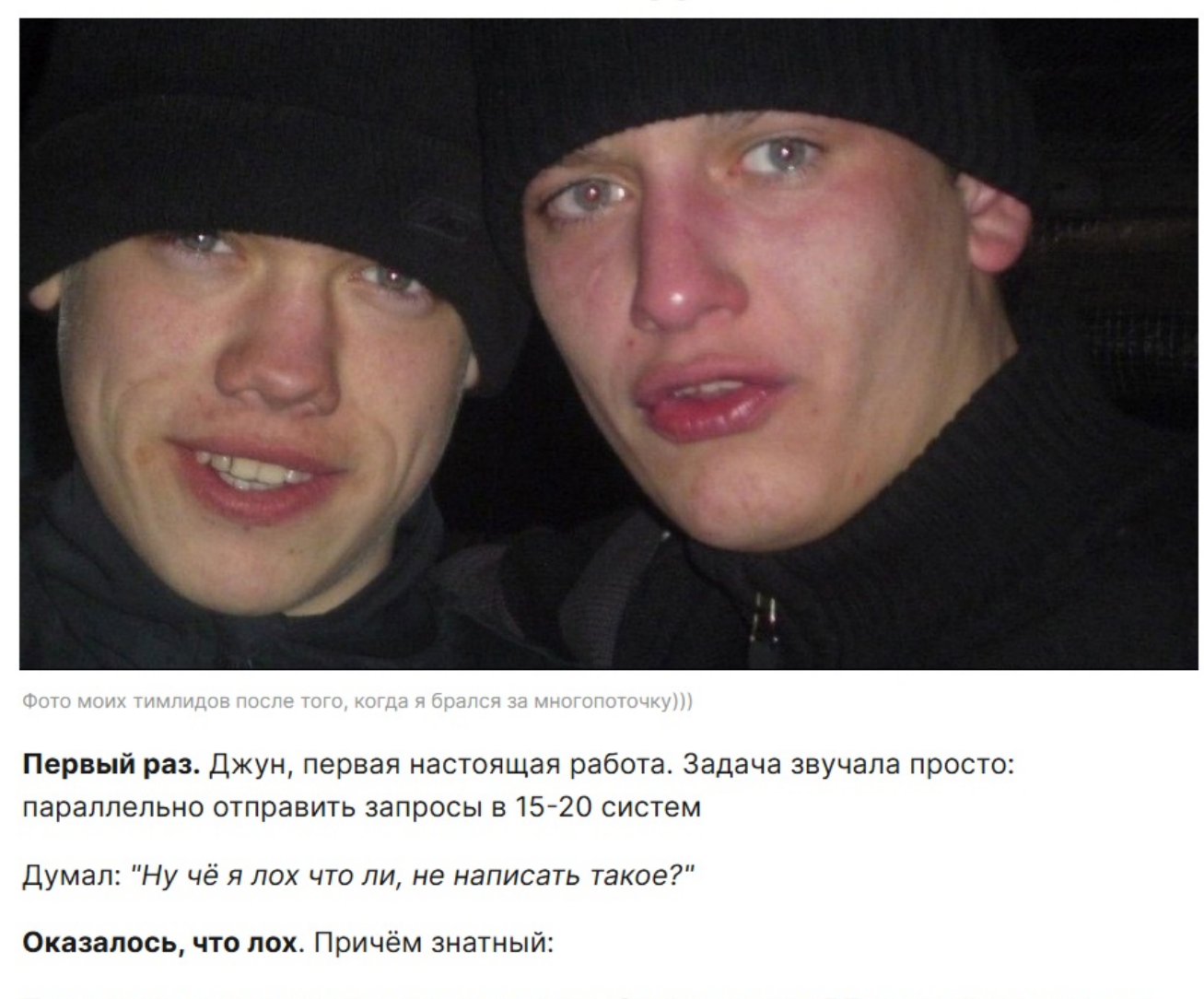


Фото моих тимлидов после того, когда я брался за многопоточку)))

Первый раз. Джун, первая настоящая работа. Задача звучала просто: параллельно отправить запросы в 15-20 систем

Думал: *"Ну чё я лох что ли, не написать такое?"*

Оказалось, что лох. Причём знатный:

Потоки начали конкурировать за данные. Один читал из БД, второй в это время перезаписывал, третий работал уже с невалидной инфой.

Иногда отваливались вообще все коннекты к базе. Несколько дней я сидел и буквально не понимал что происходит. Позвал миддла - он тоже не вкуривал несколько дней

Проблему тогда решили, и я словил главный инсайт про многопоточку, который потом повторял себе много раз:

Запустить поток реально изи. Но когда сломается - ты даже не поймёшь где

> Как предусмотреть проблему, если не знаешь что она вообще существует?

Второй раз. Я уже сеньор в крупном ФинТехе. Задача простая: кинуть 2-3 параллельных запроса во внешние системы, сагрегировать результаты, записать в базу

Сделал через `CompletableFuture`, на тестах всё работает, выкатили в прод - тоже норм. Потом нагрузка выросла, и сервис начал падать...

Несколько дней с другими сеньорами сидели и не понимали в чём дело

Потом осенило: я забыл явно указать `Executor` для `CompletableFuture`. По умолчанию он запускался на общем `ForkJoinPool`. А `ForkJoinPool` отлично работает для CPU-bound задач - когда реально что-то считаешь

Но когда ты суёшь туда долгие I/O-ожидания - запросы к внешним системам - он просто съедает общий пул, потоков на всё остальное не хватает, **и сервис начинает задыхаться**

> Вот вроде база. А сеньоры сидят несколько дней и не понимают в чём проблема

Мораль не в том, что я тупой. Мораль в том, что без знаний внутрянки и понимания где может всплыть проблема - ты не увидишь её до тех пор, пока она не случилась в проде

Что ещё хуже - многотопоточка есть везде, даже если ты её не видишь:

Глава 5. Ты уже живёшь в многопоточке. Просто не знаешь об этом

I suck at multithreading
Guys I have bad news

Есть одно стойкое заблуждение среди разработчиков: *«Я не пишу* `new Thread()` *- значит, многопоточность меня не касается»*

Касается. Ещё как:

Любой веб-сервер обрабатывает запросы конкурентно. У любой базы данных есть пул коннектов. Kafka-консьюмеры, шедулеры, кеш - **всё это постоянная конкуренция потоков за ресурсы**

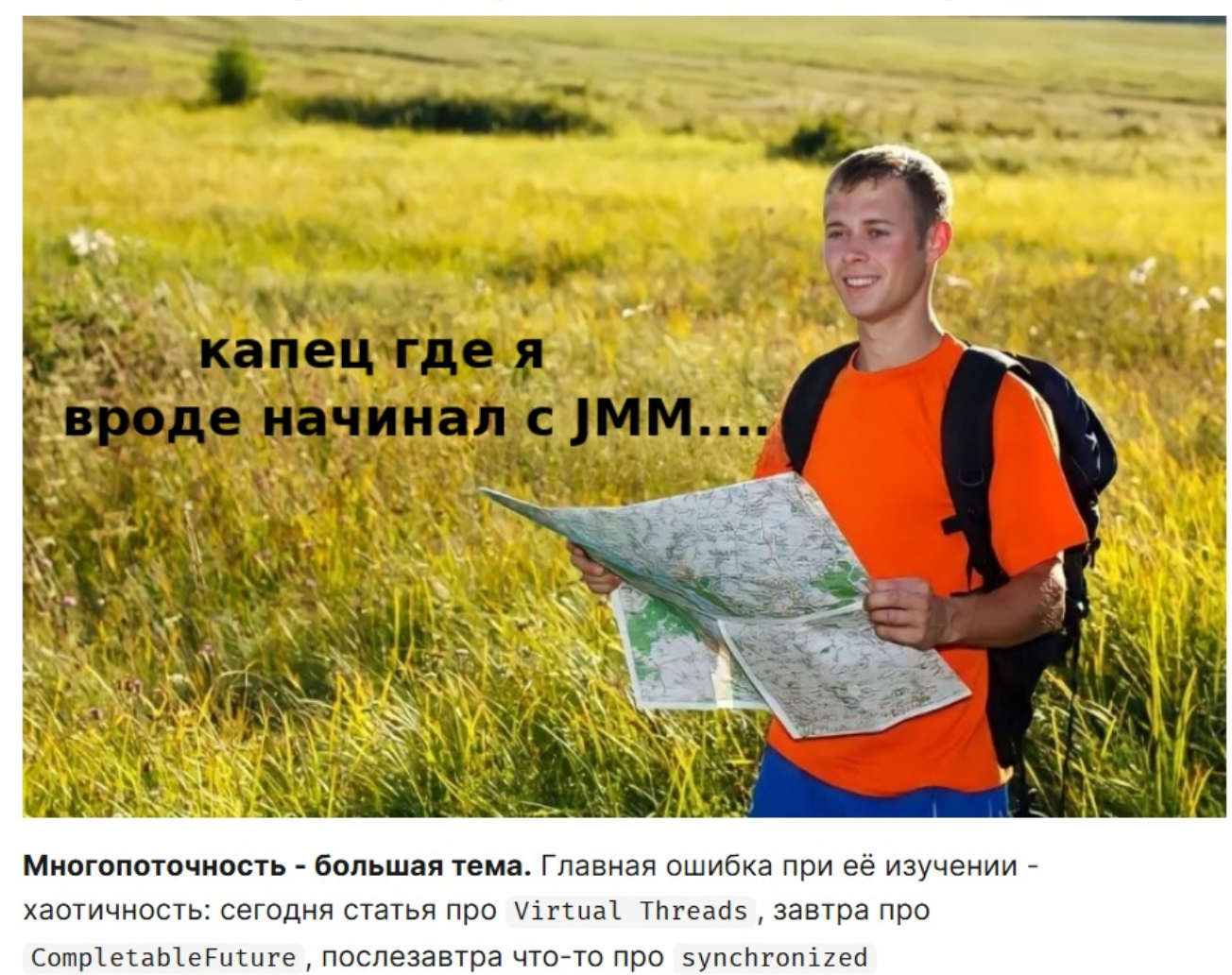
> *Под капотом огромного количества фреймворков уже преднастроена работа с потоками, но вовсе не обязательно, что она настроена идеально и тебе не придётся что-то чинить в своей компании*

Ты можешь написать самый обычный контроллер, который идеально работает на локалке - а потом в проде под нагрузкой данные начнут ломаться, запросы зависнут, и ты будешь смотреть в логи, не понимая что именно и где сломалось

Не потому что ты плохой разработчик. А потому что не видишь места, где несколько потоков одновременно лезут в одно и то же

Хорошая новость: это лечится. И лечится по понятной системе

Глава 6. Карта: что учить и в каком порядке

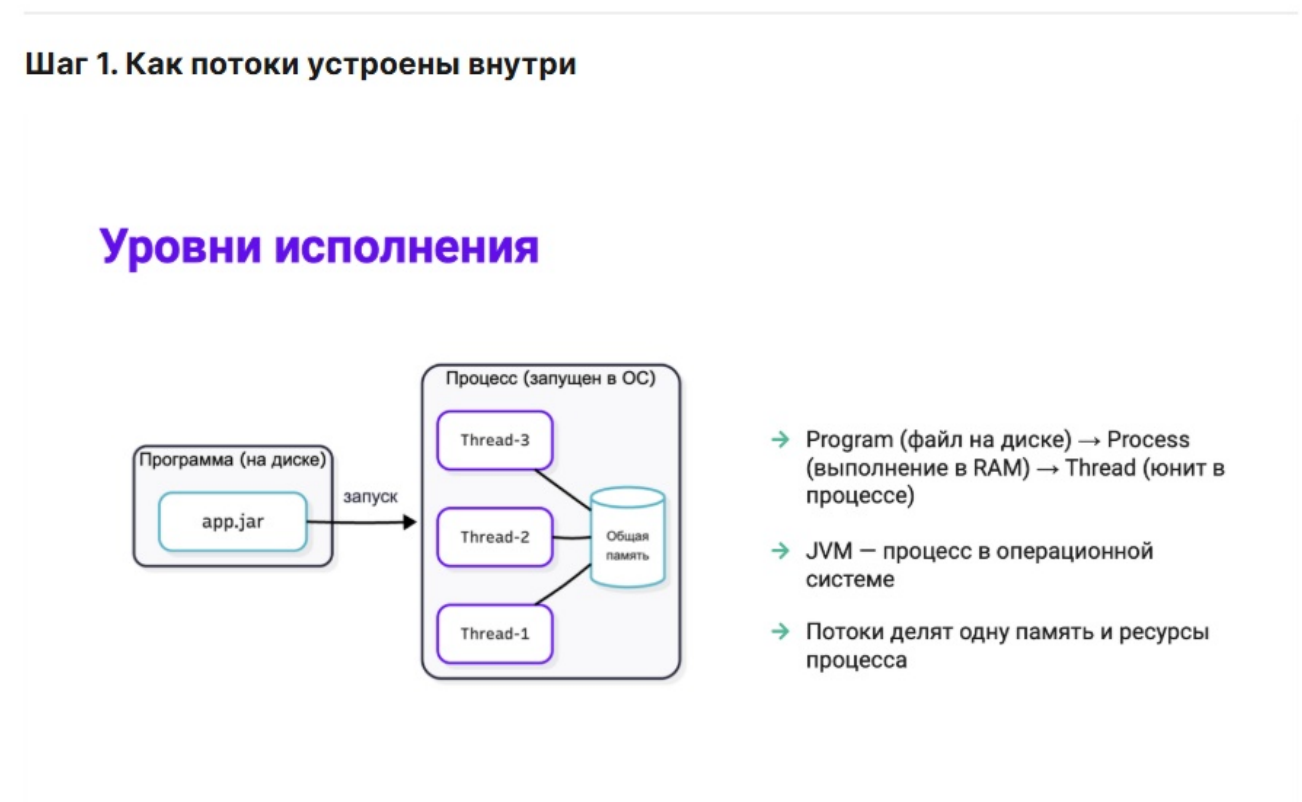

капец где я
вроде начинал с JMM....

Многопоточность - большая тема. Главная ошибка при её изучении - хаотичность: сегодня статья про `Virtual Threads`, завтра про `CompletableFuture`, послезавтра что-то про `synchronized`

В итоге каша в голове и ощущение, что тема необъятна

Но она необъятна только без системы. Вот четыре шага, где каждый строится на предыдущем:

Шаг 1. Как потоки устроены внутри


Уровни исполнения
Программа (на диске)
app.jar
запуск
Процесс (запущен в ОС)
Thread-3
Thread-2
Thread-1
Общая память
Program (файл на диске) → Process (выполнение в RAM) → Thread (юнит в процессе)
JVM — процесс в операционной системе
Потоки делят одну память и ресурсы процесса

Начинать нужно не с инструментов, а с модели: как вообще устроена память в многопоточной программе

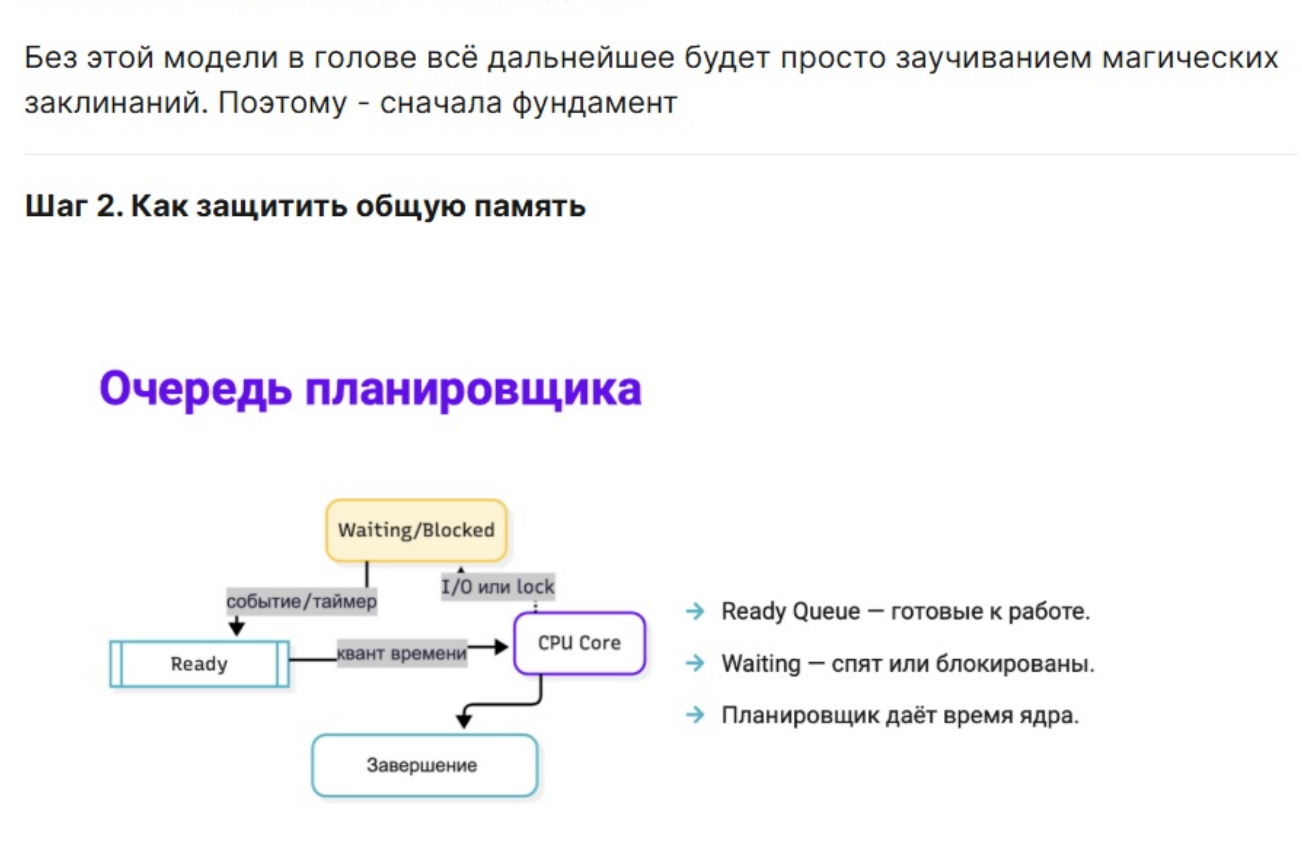
Итак, когда запускается Java-приложение, у каждого потока есть свой **стек** - личное пространство для локальных переменных. Но **heap**, где живут все объекты - общий для всех потоков одновременно

Из этого простого факта вытекает главный баг многопоточки: два потока могут одновременно обратиться к одному объекту. Один читает поле, другой в этот момент его перезаписывает. Первый получает мусор

Это, кстати, и есть **race condition** - и именно отсюда берутся баги, которые «иногда воспроизводятся, иногда нет» и которые никогда не появляются на локалке, а только в проде под нагрузкой

Без этой модели в голове всё дальнейшее будет просто заучиванием магических заклинаний. Поэтому - сначала фундамент

Шаг 2. Как защитить общую память


Очередь планировщика
Waiting/Blocked
событие/таймер
I/O или lock
Ready
квант времени
CPU Core
Завершение
Ready Queue — готовые к работе.
Waiting — спят или блокированы.
Планировщик даёт время ядра.

Здесь есть два ключевых инструмента, которые большинство путает между собой - и именно на этом валятся на собесах

1. `synchronized` - мьютекс. Гарантирует, что в один момент времени данный участок кода выполняет только один поток. Остальные ждут снаружи. Защищает от гонки

2. `volatile` - это гарантия видимости. Когда поток читает `volatile` - переменную, он получает актуальное значение из основной памяти, а не устаревшую копию из кеша процессора

Без него каждый поток может работать со своей локальной копией данных - и не знать, что другой поток давно всё изменил

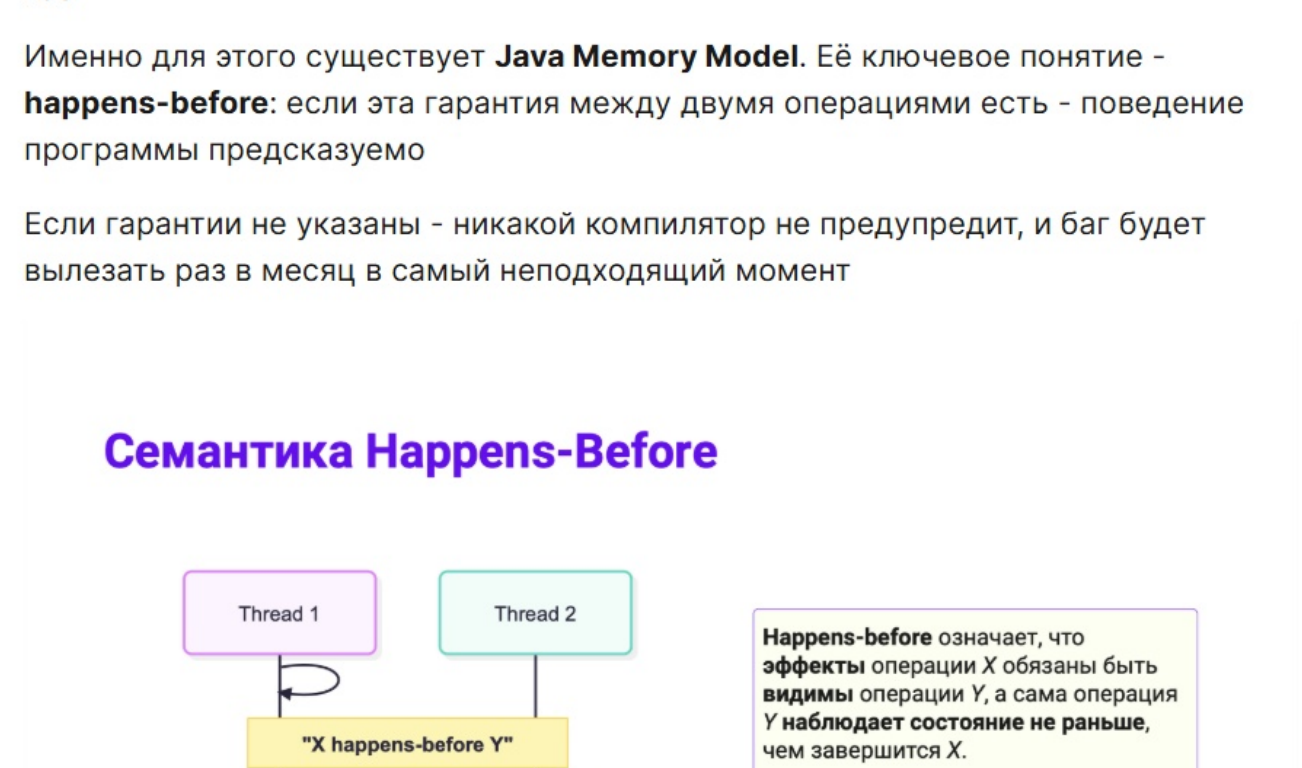
Разные инструменты для разных проблем. Путать их - ошибка уровня «не знаю базы»

Но даже правильно расставленные `synchronized` и `volatile` не дают полной защиты

Компилятор и процессор переставляют инструкции местами для оптимизации - это называется **reordering**. Для одного потока всё выглядит правильно. Для другого - хаос

Именно для этого существует **Java Memory Model**. Её ключевое понятие - **happens-before**: если эта гарантия между двумя операциями есть - поведение программы предсказуемо

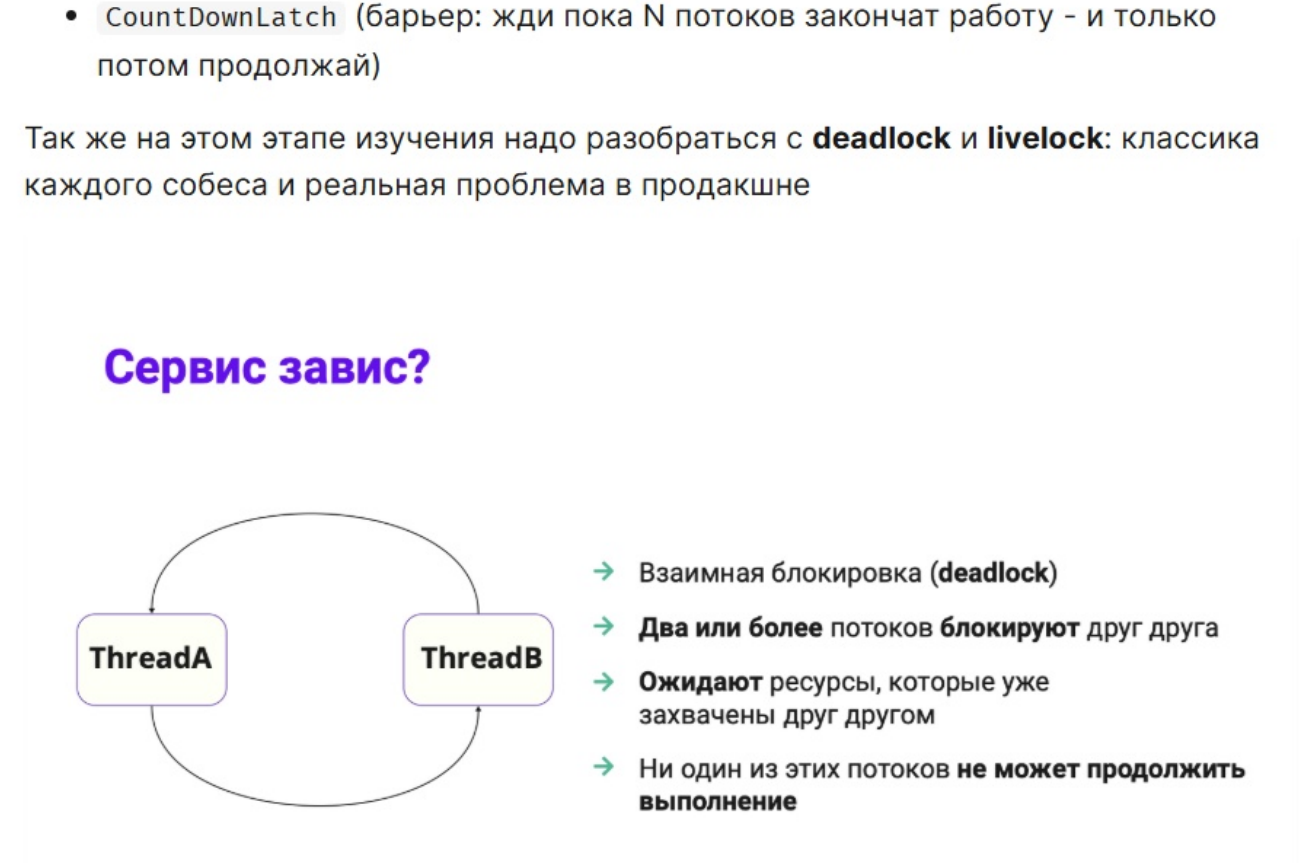
Если гарантии не указаны - никакой компилятор не предупредит, и баг будет вылезать раз в месяц в самый неподходящий момент


Семантика Happens-Before
Thread 1
Thread 2
"X happens-before Y"
Thread 1
Thread 2
Happens-before означает, что эффекты операции X обязаны быть видимы операции Y, а сама операция Y наблюдает состояние не раньше, чем завершится X.
Если X happens-before Y, то Y не увидит "полусделанных" изменений, сделанных после X.

На этом же шаге появляются продвинутые примитивы синхронизации:

- `ReentrantLock` (он гибче `synchronized` - можно прерывать ожидание и ставить таймауты)
- `Semaphore` (позволяет ограничить количество потоков, одновременно делающих одно и то же)
- `CountDownLatch` (барьер: жди пока N потоков закончат работу - и только потом продолжай)

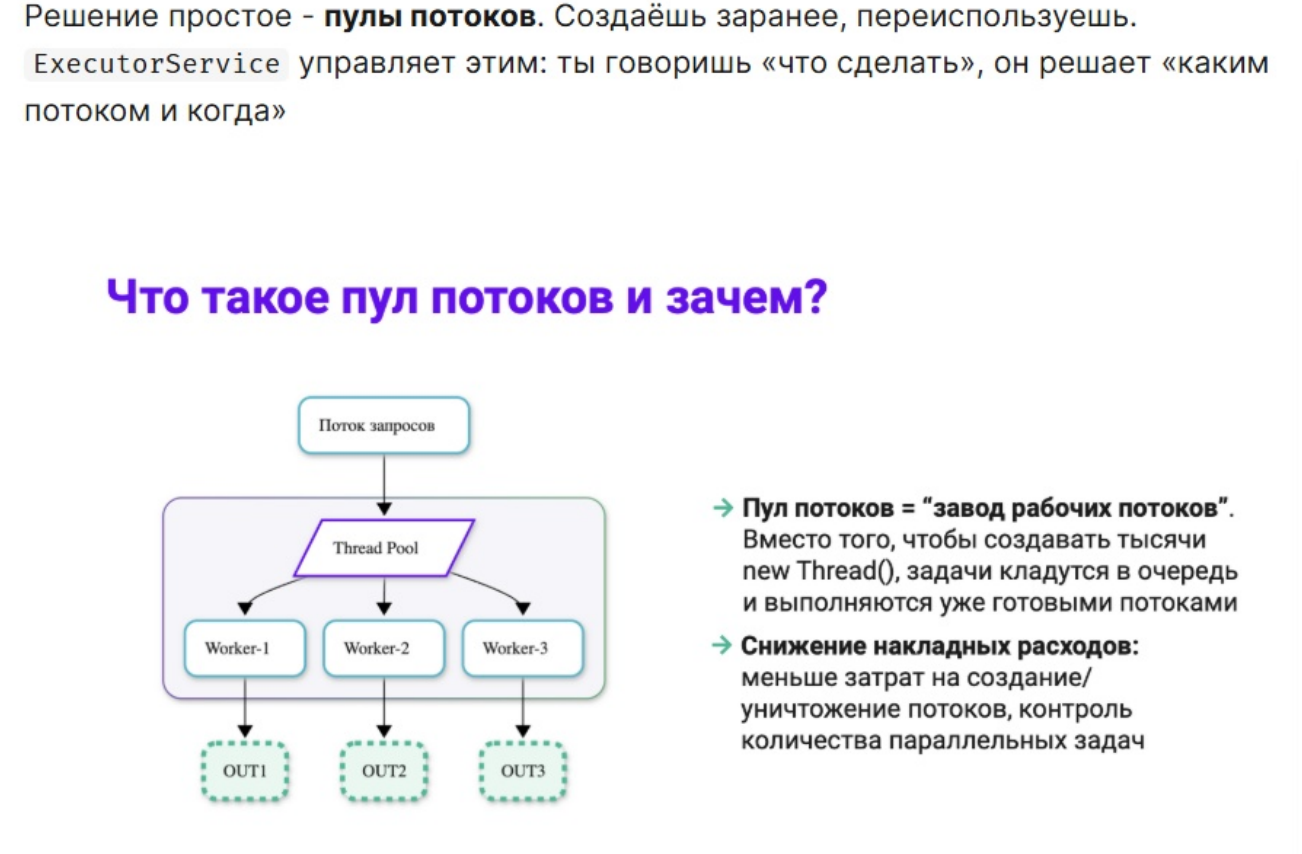
Так же на этом этапе изучения надо разобраться с **deadlock** и **livelock**: классика каждого собеса и реальная проблема в продакшне


Сервис завис?
ThreadA
ThreadB
Взаимная блокировка (deadlock)
Два или более потоков блокируют друг друга
Ожидают ресурсы, которые уже захвачены друг другом
Ни один из этих потоков не может продолжить выполнение

Шаг 3. Как правильно управлять потоками

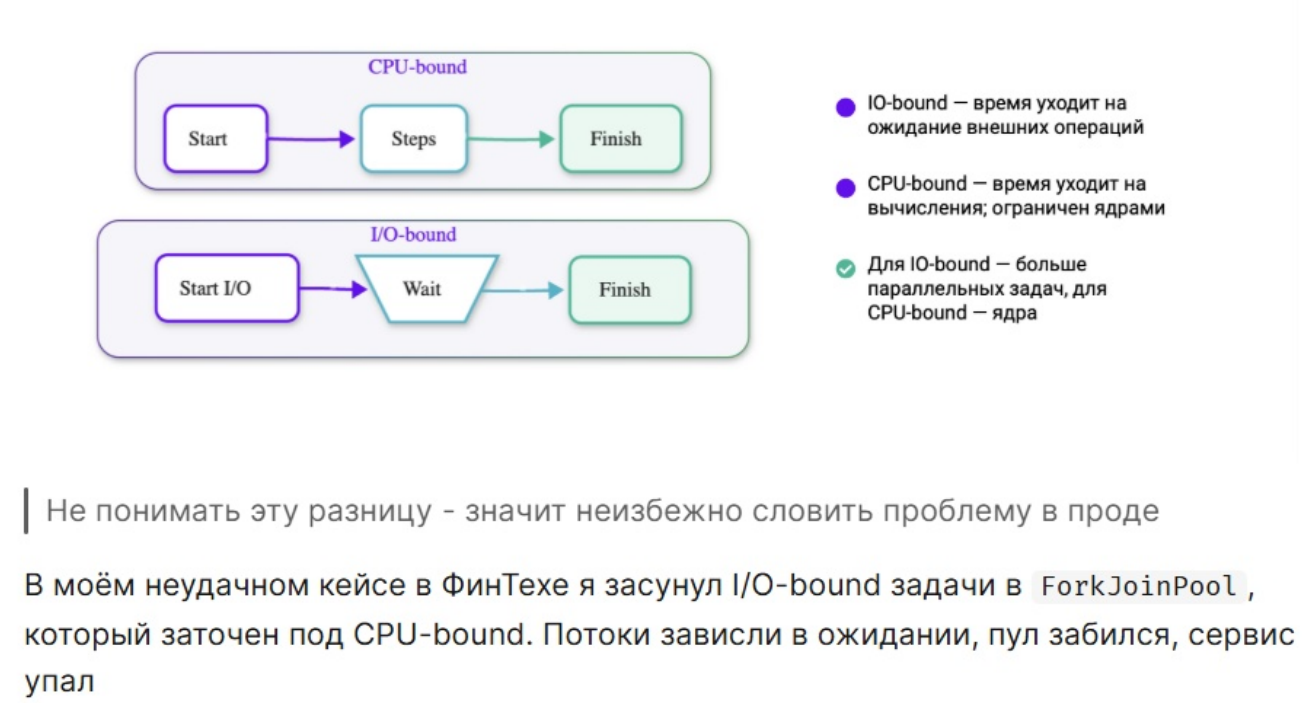
В нормальном продакшне никто не создаёт поток под каждую задачу. Поток - дорогая штука: нужна память под стек, ОС должна его планировать. Создавать потоки пачками значит либо умереть от переключений контекста, либо упереться в лимиты памяти

Решение простое - **пулы потоков**. Создаёшь заранее, переиспользуешь. `ExecutorService` управляет этим: ты говоришь «что сделать», он решает «каким потоком и когда»


Что такое пул потоков и зачем?
Поток запросов
Thread Pool
Worker-1
Worker-2
Worker-3
OUT1
OUT2
OUT3
Пул потоков = "завод рабочих потоков". Вместо того, чтобы создавать тысячи new Thread(), задачи кладутся в очередь и выполняются уже готовыми потоками
Снижение накладных расходов: меньше затрат на создание/уничтожение потоков, контроль количества параллельных задач

Ключевое, что нужно понять: правильный размер пула зависит от типа нагрузки. И тут есть два принципиально разных типа:

- **CPU-bound** - задачи, которые реально считают: криптография, сжатие, парсинг. Узкое место - процессор. Потоков нужно примерно по числу ядер. Больше - контекст-свитчи съедят весь выигрыш
- **I/O-bound** - задачи, которые в основном ждут: запросы к БД, внешние API, файловые операции. Узкое место - ожидание. Потоков можно держать больше, потому что большую часть времени они простаивают


IO-bound vs CPU-bound
CPU-bound
Start
Steps
Finish
I/O-bound
Start I/O
Wait
Finish
IO-bound — время уходит на ожидание внешних операций
CPU-bound — время уходит на вычисления; ограничен ядрами
Для IO-bound — больше параллельных задач, для CPU-bound — ядра

> Не понимать эту разницу - значит неизбежно словить проблему в проде

В моём неудачном кейсе в ФинТехе я засунул I/O-bound задачи в `ForkJoinPool`, который заточен под CPU-bound. Потоки зависли в ожидании, пул забился, сервис упал

На этом же шаге так же надо прошарить за два важных инструмента:

1. Первый - `CompletableFuture` - запустить несколько операций параллельно, дождаться всех, объединить результаты, обработать ошибки и таймауты

Типичный кейс применения: собрать ответы от трёх внешних сервисов и вернуть один агрегированный ответ пользователю

2. Второй - Virtual Threads (Java 21) - лёгкие потоки под управлением JVM, а не ОС. Можно запускать миллионами, они не съедают память как обычные потоки. При I/O-ожидании JVM автоматически освобождает реальный поток и отдаёт его другой задаче

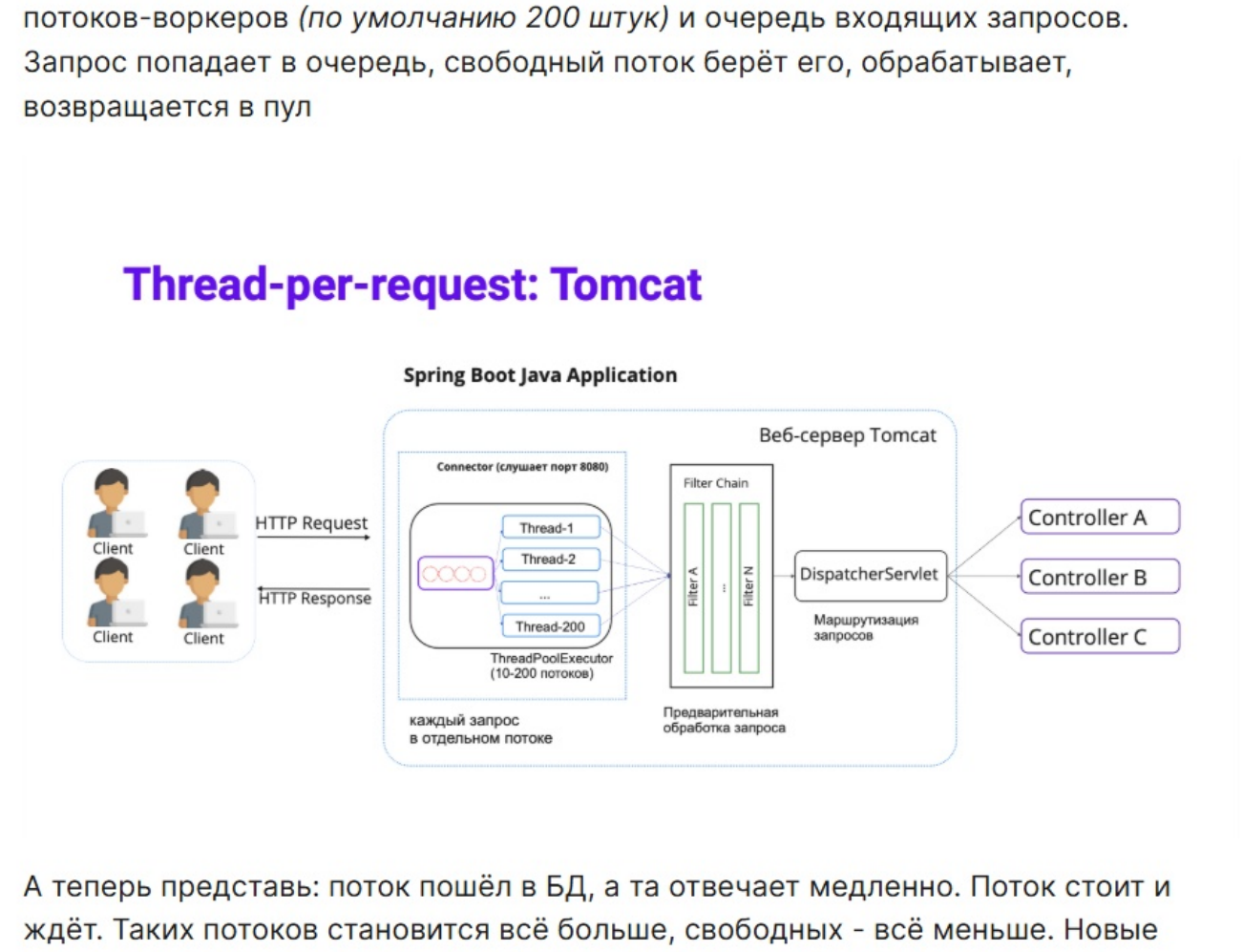
> По сути ты пишешь простой синхронный код - а работает как асинхронный. Для I/O-heavy бэкенда это огромный буст

Шаг 4. Потрогать и понять как это выглядит в реальном сервисе под нагрузкой

Последний шаг - создать проект и потрогать все технологии в работе, поймать ошибки, понять откуда они возникли, пофиксить и так далее

Вот мой простой пример, который есть почти в каждом Java-бэкенде и который ты можешь взять за основу:

У нас есть входящий HTTP-запрос - его принимает Tomcat. Внутри у него пул потоков-воркеров *(по умолчанию 200 штук)* и очередь входящих запросов. Запрос попадает в очередь, свободный поток берёт его, обрабатывает, возвращается в пул


Thread-per-request: Tomcat
Spring Boot Java Application
Веб-сервер Tomcat
Client
Client
Client
Client
HTTP Request
HTTP Response
Connector (слушает порт 8080)
Thread-1
Thread-2
...
Thread-200
ThreadPoolExecutor (10-200 потоков)
каждый запрос в отдельном потоке
Filter Chain
Filter A
Filter N
Предварительная обработка запроса
DispatcherServlet
Маршрутизация запросов
Controller A
Controller B
Controller C

А теперь представь: поток пошёл в БД, а та отвечает медленно. Поток стоит и ждёт. Таких потоков становится всё больше, свободных - всё меньше. Новые запросы копятся в очереди, latency растёт, клиенты получают таймауты и начинают ретраить

Нагрузка растёт ещё больше. CPU при этом почти пустой - сервис умирает просто потому что закончились потоки в пуле

Чини, развлекайся:)

Кроме починки, не забудь про узкие горлышки, которые нужно уметь видеть и которые должны отработать в проекте:

- **БД** - пул коннектов конечен, транзакции могут блокировать друг друга
- **Внешние сервисы** - медленно отвечают, потоки зависают в ожидании - нужно уметь в Circuit Breaker
- **Синхронизация в коде** - `synchronized` на методе, который вызывается тысячи раз в секунду, и все потоки выстраиваются в очередь на вход. Пока один поток внутри, остальные стоят и ждут. Решай проблему:)

Думаю на этом шаге становится понятно, зачем нужна вся предыдущая цепочка:

- Не знаешь как устроены пулы? Видишь пустой CPU и не понимаешь что происходит
- Не знаешь JMM? Данные кривые, баг раз в месяц, воспроизвести невозможно
- и так далее

Потому что если ты понимаешь всё от фундамента до продакшна, то находишь проблему за минуты. Не понимаешь - тыкаешь пальцем в небо

> Вот и вся карта. Четыре шага, каждый строится на предыдущем

Что дальше?

Окей, примерная карта изучения многопоточки у тебя есть

Единственное что хочу добавить - про прикладное применение технологий. Многие, получив такую карту, идут гуглить каждый пункт отдельно

Это работает, я сам так делал. Но есть один момент который сложно закрыть статьями: **практика на реальной нагрузке**

- **Race condition** ты не почувствуешь, пока не напишешь код который сломается под нагрузкой у тебя на глазах
- **Deadlock** не запомнишь, пока не поймаешь его сам и не поймёшь почему он там вообще возник
- *ну и так далее*

Именно поэтому я в итоге собрал курс, где финальный проект - не «реализуй связный список», а продакшн-система, которую нужно сначала написать, потом сломать нагрузкой, потом починить:

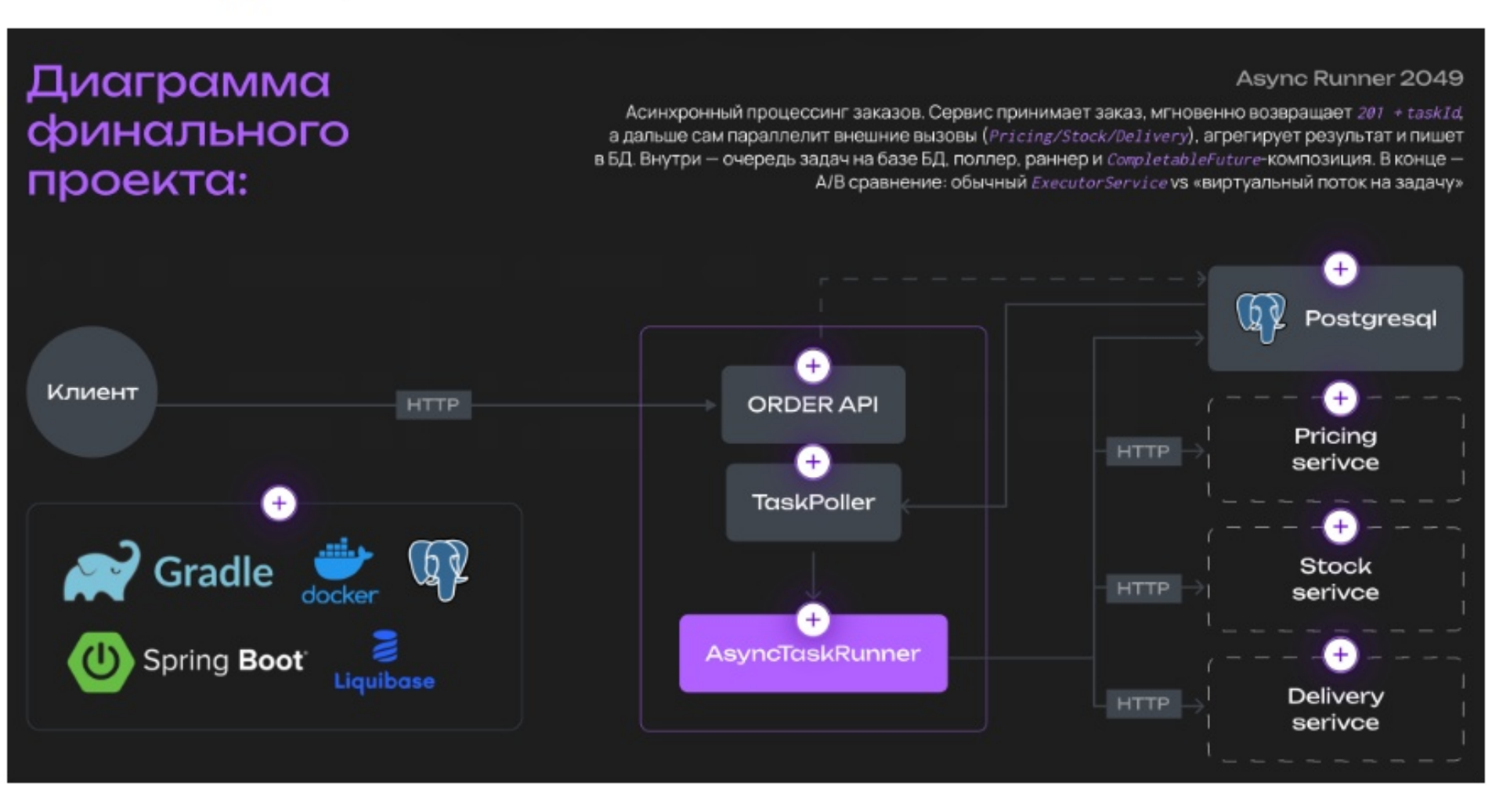


Диаграмма финального проекта:
Async Runner 2049
Асинхронный процессинг заказов. Сервис принимает заказ, мгновенно возвращает 201 + taskId, а дальше сам параллелит внешние вызовы (Pricing/Stock/Delivery), агрегирует результат и пишет в БД. Внутри — очередь задач на базе БД, поллер, раннер и CompletableFuture-композиция. В конце — A/B сравнение: обычный ExecutorService vs «виртуальный поток на задачу»
Клиент
HTTP
ORDER API
TaskPoller
AsyncTaskRunner
Postgresql
HTTP
Pricing serivce
HTTP
Stock serivce
HTTP
Delivery serivce
Gradle
docker
Spring Boot
Liquibase

Тот самый production-кейс из которого берутся баги вроде моего факапа в синем ФинТехе

> **Сайт курса с программой, форматом, длительностью, бонусами, тарифами и стоимостью - sorokin.school/multithreading**

Ниже я расскажу про продукт подробнее своими словами, и даже если ты *пока* уверен, что разберешься сам, всё равно советую почитать, чтоб подробнее понять какие штуки учить

Программа

Модуль 1 — Открываем капот

Ты наконец поймёшь что происходит внутри когда запускается твоё Java-приложение — как устроены процессы, потоки, общая память, как планировщик ОС решает кому дать время на CPU, а так же:

- **Увидишь** полный жизненный цикл потока от NEW до TERMINATED, разберёшься как его создать, запустить и корректно остановить
- **Поймёшь** почему порядок выполнения потоков непредсказуем, и что с этим реально делать
- **Потрогаешь** первые инструменты синхронизации: `synchronized` и `volatile` - не просто послушаешь «что это такое», а когда что использовать и почему их путают даже опытные разработчики

После этого модуля у тебя появятся основы в голове. Ты будешь видеть что происходит в многопоточном коде, а не гадать

Модуль 2 — Укрощаем хаос

Ты разберёшься с тем, из-за чего люди реально сыпятся на собесах и в проде, а так же:

- **Поймёшь** Java Memory Model - когда именно один поток гарантированно видит изменения от другого, и почему без этого знания код может выглядеть правильно, проходить все тесты и ломаться раз в месяц в проде
- **Разберёшься** с happens-before и reordering - почему компилятор переставляет твои инструкции местами и как это тихо ломает логику
- **Освоишь** продвинутые примитивы: `ReentrantLock`, `Semaphore`, `ReadWriteLock`, `CountDownLatch` и и поймёшь в каких ситуациях они нужны вместо простого `synchronized`
- **Напишешь** свою `ArrayBlockingQueue` с нуля на `synchronized` и `wait/notify` - именно такую, которая работает под капотом в пулах потоков и Tomcat'е

На десерт ты своими руками поймаешь `deadlock`, разберёшься почему он возник и починишь его

После этого модуля `race condition` и `deadlock` - это не загадка, а понятные проблемы с понятными решениями

Модуль 3 — Инструменты продакшна

Ты освоишь инструменты, которыми пользуются в реальных проектах каждый день, а так же:

- **Разберёшься** с `ExecutorService` и пулами потоков - Fixed, Cached, Scheduled. Тут ты поймёшь какой тип пула когда брать и почему неправильный выбор кладёт сервис в проде
- **Поймёшь** как устроен `ForkJoinPool` и почему туда нельзя совать I/O-задачи
- **Научишься** строить асинхронные цепочки через `CompletableFuture`: параллельный запуск операций, объединение результатов, обработка ошибок и таймаутов
- **Разберёшься** с конкурентными коллекциями - поймёшь почему обычный `HashMap` ломается под нагрузкой и что использовать вместо него
- И наконец **разберёшься** с Virtual Threads из Java 21: как они устроены, когда дают реальный выигрыш, а где у них ограничения

После этого модуля ты будешь не угадывать инструмент, а осознанно выбирать нужный

Модуль 4 - Боевое крещение

Ты поймёшь как **Spring Boot** и **Tomcat** обрабатывают параллельные запросы под капотом, где возникают узкие места и почему сервис может умирать при почти нулевом CPU. **А потом реализуешь это сам**

> Ты напишешь асинхронный обработчик заказов: запрос приходит - сервис мгновенно отвечает «принято» и выдаёт `taskId`

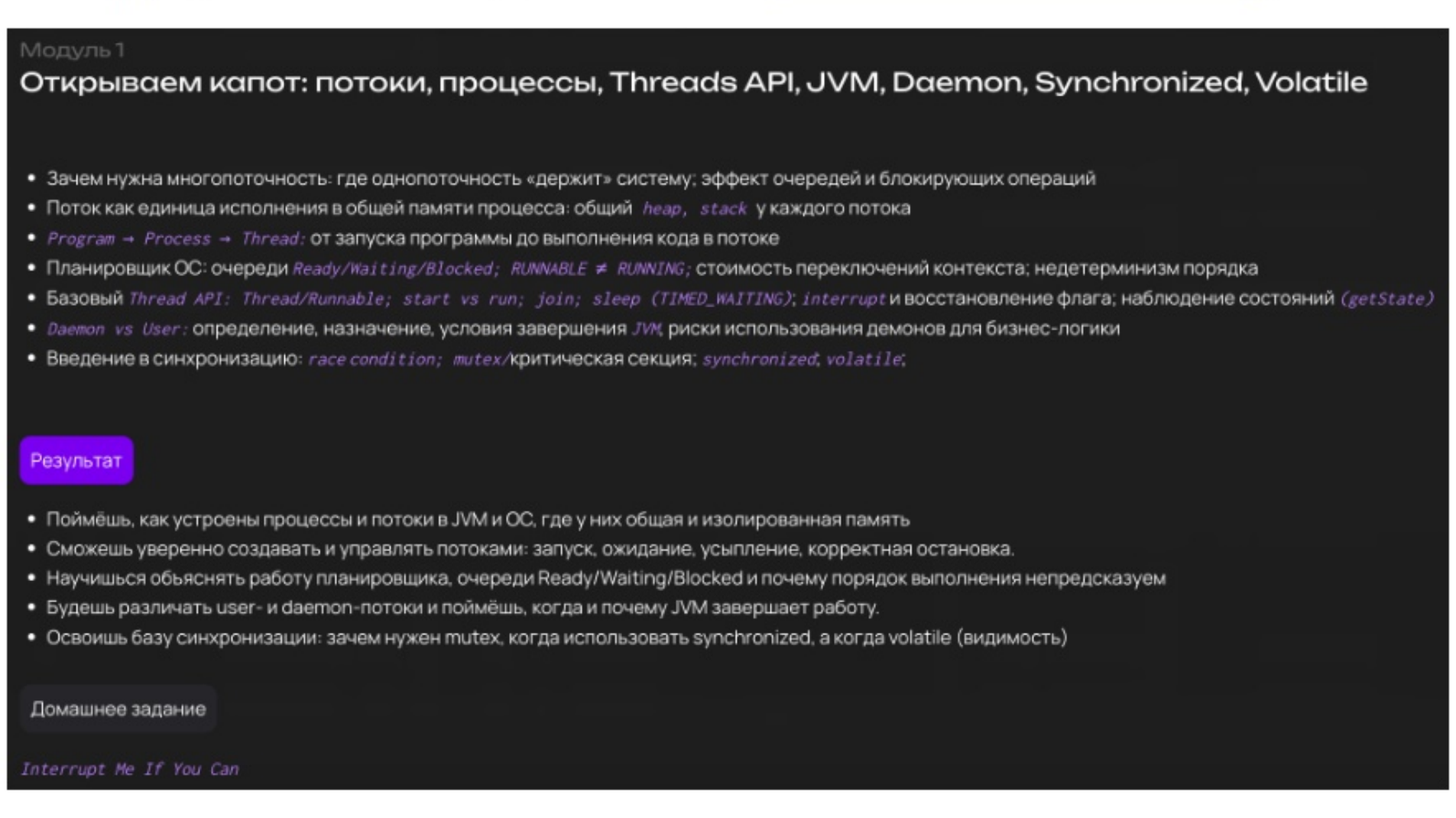
Дальше задача уходит в очередь, воркеры её подбирают, параллельно дёргают три внешних API - Stock, Pricing, Shipping - агрегируют результаты, обновляют статус: `NEW → IN_PROGRESS → DONE / FAILED / ERROR`

Потом ты сам сломаешь этот сервис. Прогонишь нагрузку, увидишь где забивается очередь, где заканчиваются потоки, где упираешься в пул коннектов. Найдёшь узкое место и починишь

> На выходе у тебя будет рабочий микросервис который держит 10.000 RPS на стеке Java 21+, Spring Boot, PostgreSQL, Docker Compose с нагрузочными тестами через k6 и наблюдаемостью через Grafana

Не TODO List, а реальный production-кейс, который ляжет на GitHub и будет работать на тебя долгие годы

Супер подробную информацию по каждому модулю, финальному проекту и тому, что внутри, ты опять же найдешь на сайте - sorokin.school/multithreading



Модуль 1

Открываем капот: потоки, процессы, Threads API, JVM, Daemon, Synchronized, Volatile

- Зачем нужна многопоточность: где однопоточность «держит» систему; эффект очередей и блокирующих операций
- Поток как единица исполнения в общей памяти процесса: общий `heap, stack` у каждого потока
- `Program → Process → Thread:` от запуска программы до выполнения кода в потоке
- Планировщик ОС: очереди `Ready/Waiting/Blocked; RUNNABLE ≠ RUNNING;` стоимость переключений контекста; недетерминизм порядка
- Базовый `Thread API: Thread/Runnable; start vs run; join; sleep (TIMED_WAITING); interrupt` и восстановление флага; наблюдение состояний `(getState)`
- `Daemon vs User:` определение, назначение, условия завершения `JVM;` риски использования демонов для бизнес-логики
- Введение в синхронизацию: `race condition; mutex/`критическая секция; `synchronized, volatile;`

Результат

- Поймёшь, как устроены процессы и потоки в JVM и ОС, где у них общая и изолированная память
- Сможешь уверенно создавать и управлять потоками: запуск, ожидание, усыпление, корректная остановка.
- Научишься объяснять работу планировщика, очереди Ready/Waiting/Blocked и почему порядок выполнения непредсказуем
- Будешь различать user- и daemon-потоки и поймёшь, когда и почему JVM завершает работу.
- Освоишь базу синхронизации: зачем нужен mutex, когда использовать synchronized, а когда volatile (видимость)

Домашнее задание

`Interrupt Me If You Can`

Вот настолько там подробно описана внутрянка модулей:)

Теперь к формату курса:

1. Ты смотришь записанные видео-уроки на GetCourse когда удобно - ставишь на паузу, пересматриваешь, возвращаешься к нужному месту

2. Внутри 13 часов контента без воды - большинство проходят за +- 1 месяц в своём темпе

3. 60% времени ты видишь как я пишу код и повторяешь за мной, 40% времени я на понятных визуальных схемах объясняю что происходит внутри

4. Внутри так же 4 практические домашки по нарастающей - от простых потоков до продакшн-кейса под нагрузкой

Кроме самого курса, ты так же бесплатно получаешь 4 бонуса:

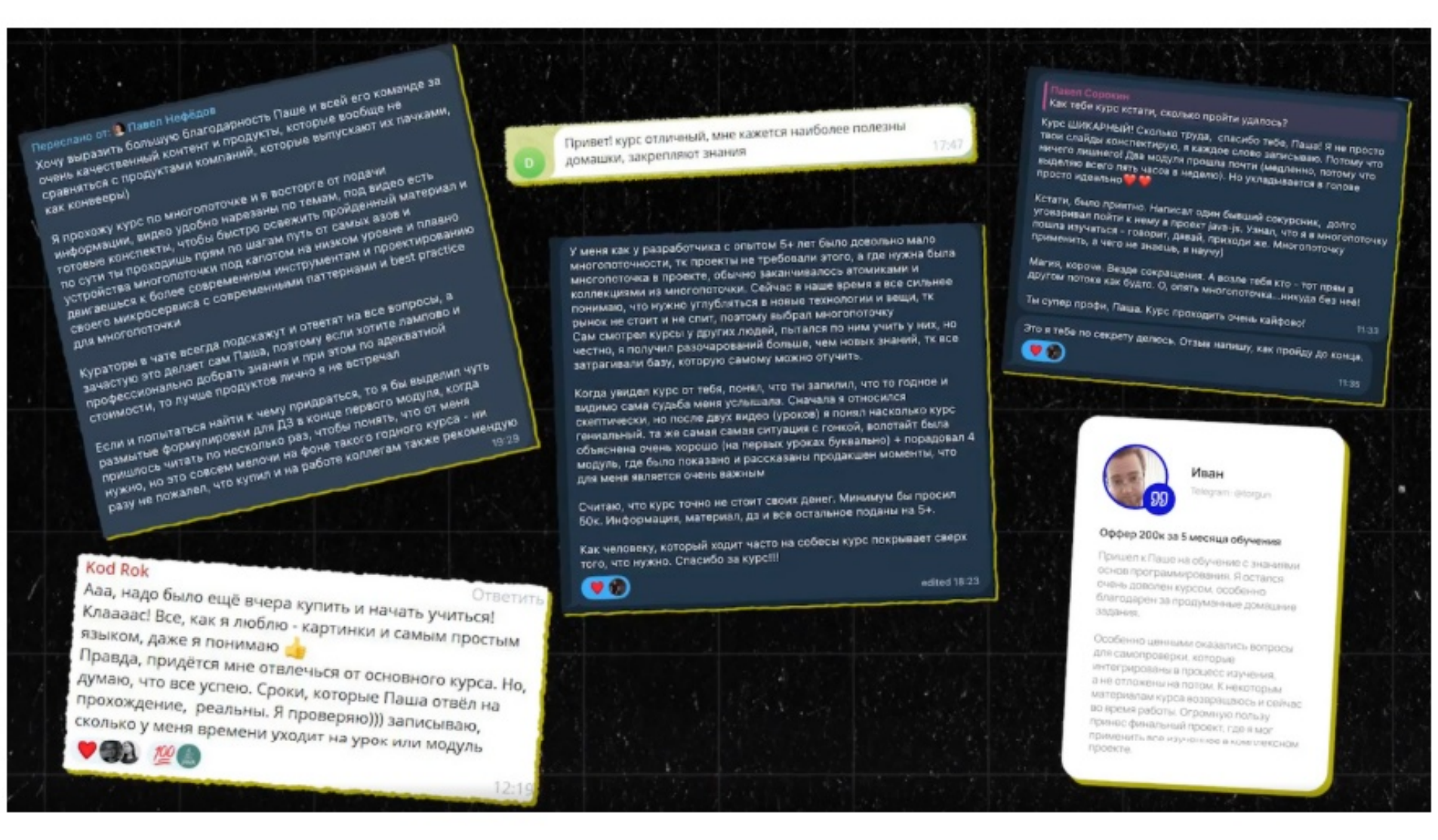
1. **Закрытый чат «Поток многопоточности»** - ты получаешь доступ в Telegram-чат учеников с менторской поддержкой. Застрял на домашке - задаёшь вопрос и получаешь ответ. Не бойся, тут точно не будет такого, что ты чего-то не понял
2. **PDF: ТОП-5 примеров многопоточности в реальной работе** - ты забираешь готовый рабочий код: потокобезопасный кеш, rate limiter, producer-consumer на BlockingQueue, примеры с Virtual Threads - с комментариями «почему именно так». Берёшь и используешь в своих проектах
3. **Чек-лист «20 ошибок в многопоточности, которые убивают проекты»** - ты получаешь шпаргалку с разбором типичных косяков которые допускают даже опытные разработчики. Держишь под рукой при код-ревью. На собесах часто дают код с багом и просят найти проблему - с этим чек-листом ты видишь ошибки сразу
4. **Видео-разбор «Топ-10 вопросов про многопоточность на собеседованиях»** - ты смотришь 1.5 часа живого разбора: как отвечать на самые частые вопросы, какие формулировки работают, как вести себя если не знаешь ответа, как не попасться в ловушки интервьюера. Даже если всё понял из курса - уметь объяснить это отдельный навык

Тарифы

А вот про их наполнение и стоимость уже читай на сайте, если ты вдруг до сих пор на него не перешёл - sorokin.school/multithreading

> Я периодически добавляю всякие плюшки в тарифы и меняю стоимость, поэтому в статье не могу честно указать наполнение и ценник

Что говорят про курс те, кто уже прошёл



Привет! курс отличный, мне кажется наиболее полезны домашки, закрепляют знания

Kod Rok
Ааа, надо было ещё вчера купить и начать учиться! Клаааас! Все, как я люблю - картинки и самым простым языком, даже я понимаю 👍
Правда, придётся мне отвлечься от основного курса. Но, думаю, что все успею. Сроки, которые Паша отвёл на прохождение, реальны. Я проверяю))) записываю, сколько у меня времени уходит на урок или модуль

Ключевая ценность которую отмечают ученики - не просто знания, а их полнота и системность. Люди которые годами откладывали эту тему закрывали её за 2 недели

> И да, некоторые пишут «я бы даже больше заплатил». Так что вопросы по цене не ко мне)))

Вот и всё

Хватит откладывать эту тему. Хватит закрывать вакансии со вздохом потому что там написано «знание многопоточности». Хватит обходить стороной задачи с `concurrency`

За месяц ты можешь закрыть то, что мешало годами

> Выбрать тариф и начать - sorokin.school/multithreading

P.S. Если остались вопросы - пиши в личку, отвечу в течение пары часов
t.me/pave1ss

---

5 марта, 01:17 · 1027 просмотров

Поделиться